

Guide utilisateur

Dilemme du prisonnier

Documentation fonctionnelle vérifiée au 19 juin 2026

Application : environnement d'expérimentation de stratégies Rhai

Version du document : 2.0

État de référence : application au 19 juin 2026

Public : utilisateurs de l'interface web

Sommaire

- 1 Objet de l'application
- 2 Accéder à l'application
- 3 Lancer un premier tournoi Classic
- 4 Sélectionner et gérer les stratégies
- 5 Écrire une stratégie Rhai
- 6 Configurer le mode Classic
- 7 Configurer le mode Multi
- 8 Lire les résultats
- 9 Dépannage

1. Objet de l'application

L'application permet de créer des stratégies qui choisissent de coopérer ou de trahir, puis de les comparer dans deux modes de tournoi.

Classic Exécution : chaque paire de stratégies joue un nombre fixe de tours

Paramètre : nombre d'itérations et matrice de gains

Résultats : classement, victoires/nuls/défaites, parties, matrice et progression des scores

Multi Exécution : toutes les stratégies produisent des actions en parallèle pendant une durée fixe

Paramètre : durée en secondes

Résultats : classement, score, nombres de coopérations et de trahisons

Les stratégies sont propres à un mode. Le passage de Classic à Multi change donc la liste proposée et vide la sélection courante.

Langue de l'interface. La majorité des commandes et résultats sont actuellement affichés en anglais. Ce guide reprend leur libellé exact en gras.

2. Accéder à l'application

Avec la configuration fournie dans le dépôt, ouvrir :

```
http://localhost:5173
```

Si l'application est déployée ailleurs, utiliser l'adresse communiquée par l'administrateur.

Démarrage local avec Docker

Prérequis : Docker et Docker Compose.

Depuis la racine du projet :

```
cp .env.example .env
docker compose up --build
```

Attendre le démarrage des trois services (`postgres`, `backend`, `frontend`), puis ouvrir l'adresse ci-dessus. Pour arrêter les services sans effacer les données :

```
docker compose down
```

Le bouton soleil/lune, en haut à droite, change le thème de l'interface et de l'éditeur.

3. Lancer un premier tournoi Classic

1. Conserver l'onglet Classic.
2. Dans Select strategies, cliquer sur au moins deux cartes. Les stratégies retenues apparaissent sous forme d'étiquettes.
3. Conserver Number of iterations à 200 pour un premier essai.
4. Conserver les gains proposés : Temptation 5, Reward 3, Punishment 1, Sucker 0.
5. Cliquer sur Launch tournament.
6. Attendre le message Tournament completed successfully!.

Pendant l'exécution, une entrée En cours apparaît dans Tournament history. À la fin, le panneau inférieur charge les résultats du tournoi créé.

Le bouton de lancement reste désactivé tant que moins de deux stratégies sont sélectionnées ou qu'un tournoi est en cours de calcul.

4. Sélectionner et gérer les stratégies

Rechercher et sélectionner

- Search strategy... filtre les stratégies par nom, sans tenir compte des majuscules.
- Un clic sur une carte l'ajoute au tournoi.
- La croix d'une étiquette retire la stratégie de la sélection sans la supprimer.
- Une stratégie déjà sélectionnée disparaît de la liste disponible.

Créer une stratégie

1. Choisir d'abord l'onglet Classic ou Multi correspondant au futur usage.
2. Cliquer sur Add a strategy.
3. Renseigner Name, Description et Script Rhai.
4. Vérifier Type de tournoi. Ce choix détermine les variables disponibles et la liste dans laquelle la stratégie sera visible.
5. Corriger toute erreur affichée sous l'éditeur.
6. Cliquer sur Save.

Les trois champs texte sont obligatoires. Le serveur limite le nom à 255 caractères. Les blocs de droite insèrent des exemples dans l'éditeur ; Help décrit les variables du mode sélectionné.

Modifier ou supprimer

Survoler une carte :

- le crayon ouvre la stratégie en modification ;
- la corbeille ouvre une confirmation de suppression.

La suppression d'une stratégie entraîne aussi, par le fonctionnement actuel de la base, la suppression des tournois auxquels elle participait. Cette action est définitive.

5. Écrire une stratégie Rhai

Le script est exécuté dans une fonction fournie par le moteur. Il ne faut pas déclarer de fonction principale. Chaque exécution doit retourner l'une de ces deux valeurs :

```
return Choice::COOPERATE;
return Choice::BETRAY;
```

La valeur interne d'une coopération est 0 et celle d'une trahison est 1. Pour comparer un choix passé, utiliser de préférence `Choice::COOPERATE.value` ou `Choice::BETRAY.value`.

L'éditeur repère certaines erreurs simples (délimiteurs, chaîne ou commentaire non fermé, opérateur `==`). À l'enregistrement, le backend exécute une validation Rhai supplémentaire. Une boucle excessive peut être interrompue par la limite d'instructions du moteur.

Variables du mode Classic

Variable exacte	Contenu
<code>last_current_strokes</code>	choix passés de la stratégie courante
<code>last_opposing_strokes</code>	choix passés de l'adversaire
<code>cost_cooperate</code>	gain d'une coopération mutuelle
<code>cost_betray</code>	gain d'une trahison mutuelle
<code>cost_betray_cooperate</code>	gain lorsque la stratégie trahit et l'adversaire coopère
<code>cost_cooperate_betray</code>	gain lorsque la stratégie coopère et l'adversaire trahit

Le nom `strokes` est bien celui exposé par l'application ; ne pas le corriger en `strokes` dans un script.

Exemple donnant-donnant :

```
if len(last_opposing_strokes) == 0 {
    return Choice::COOPERATE;
} else if last_opposing_strokes[-1] == Choice::COOPERATE.value {
    return Choice::COOPERATE;
} else {
    return Choice::BETRAY;
}
```

Variables du mode Multi

Variable exacte	Contenu
last_strokes	liste chronologique globale des actions déjà produites
strategies_strokes_locations	pour chaque stratégie, positions de ses actions dans last_strokes
own_number	indice de la stratégie courante dans le tournoi

Exemple : répéter sa propre dernière action, ou coopérer au départ.

```
let my_indices = strategies_strokes_locations[own_number];
if len(my_indices) == 0 {
    return Choice::COOPERATE;
}
let my_last_choice = last_strokes[my_indices[-1]];
if my_last_choice == Choice::BETRAY.value {
    return Choice::BETRAY;
}
return Choice::COOPERATE;
```

Le bloc Tit for tat (Multi) fourni par l'interface suppose exactement deux stratégies (1 - own_number). Il ne doit pas être utilisé tel quel avec trois stratégies ou plus.

6. Configurer le mode Classic

Nombre d'itérations

Number of iterations est le nombre de tours joué par chaque paire. La valeur initiale est 200. Le champ indique une plage de 1 à 10 000.

Avec n stratégies, le tournoi crée $n \times (n - 1) / 2$ parties. Quatre stratégies produisent donc six parties ; avec 200 itérations, cela représente 1 200 tours de confrontation.

Matrice de gains

Situation vue par une stratégie	Champ	Défaut
elle trahit, l'autre coopère	Temptation	5
les deux coopèrent	Reward	3
les deux trahissent	Punishment	1
elle coopère, l'autre trahit	Sucker	0

Les champs indiquent une plage de -10 à 20. La matrice affichée sous les champs permet de vérifier le score des deux participants dans chaque situation.

L'application n'impose pas l'ordre mathématique du dilemme du prisonnier. Pour la configuration classique, conserver Temptation > Reward > Punishment > Sucker, soit 5 > 3 > 1 > 0 par défaut.

7. Configurer le mode Multi

1. Ouvrir l'onglet Multi. La sélection Classic est alors effacée.
2. Sélectionner au moins deux stratégies de type Multi.
3. Régler Duration (seconds). La valeur initiale est 10 et le champ indique une plage de 1 à 3 600 secondes.
4. Cliquer sur Launch tournament et attendre la fin de la durée choisie.

Dans ce mode, il n'y a ni parties deux à deux, ni matrice de gains configurable. Chaque stratégie s'exécute dans son propre thread et ajoute autant d'actions que possible au flux global jusqu'à la fin du chronomètre.

Le score affiché est calculé après le tournoi à partir du nombre de coopérations et de trahisons de chaque stratégie. La valeur d'une trahison vaut cinq fois celle d'une coopération ; la valeur de base dépend des volumes globaux d'actions. En conséquence, le score dépend aussi du nombre d'actions produites, qui peut varier selon le coût d'exécution du script et l'ordonnancement de la machine. Pour une comparaison exploitable, utiliser la même durée, la même machine et des scripts de complexité comparable.

8. Lire les résultats

Historique

Tournament history liste les tournois du plus récent au plus ancien. Chaque entrée affiche son numéro, sa date et la meilleure stratégie enregistrée. Un clic recharge ses détails. Les tournois Classic et Multi figurent dans la même liste.

Résultats Classic

- Results : classement par score total décroissant et bilan W / D / L (victoires, nuls, défaites).
- Matches : chaque paire, ses deux scores cumulés et le vainqueur mis en évidence.
- Matrix : score de la stratégie en ligne contre celle en colonne ; bleu pour une victoire, gris pour un nul, rouge pour une défaite.
- Insights : score cumulé de chaque stratégie à chaque numéro d'itération, additionné sur toutes ses parties.

Le score total est la somme des gains obtenus à chaque tour. Un meilleur score signifie uniquement que la stratégie a obtenu le meilleur résultat dans cette population, avec cette durée ou ce nombre d'itérations et ces paramètres.

Le mode Multi affiche une table unique : rang, stratégie, score, nombre de coopérations et nombre de trahisons. Il n'affiche pas de parties, de matrice ni de courbe d'évolution.

9. Dépannage

La page ne s'ouvre pas vérifier que les services sont démarrés et que l'adresse/port correspond à `.env`

Launch tournament est désactivé sélectionner au moins deux stratégies ; attendre la fin du tournoi en cours

Une stratégie attendue n'apparaît pas vérifier l'onglet Classic/Multi et le type enregistré pour la stratégie

Save est désactivé ou le script est refusé remplir les trois champs, corriger l'erreur Rhai et retourner un membre de `Choice`

Le tournoi échoue lire le message affiché ; vérifier les scripts, réduire la durée ou les itérations, puis vérifier le backend

Un tournoi est très long en Classic, réduire les stratégies ou itérations ; en Multi, réduire la durée

L'historique est vide lancer un tournoi ou vérifier que la base PostgreSQL utilisée est la bonne

Le frontend ne joint pas l'API vérifier `VITE_API_URL`, puis reconstruire le frontend car les variables Vite sont intégrées à la compilation

Diagnostic administrateur :

```
docker compose ps
docker compose logs backend postgres frontend
curl http://localhost:8000/health
```

Référence fonctionnelle utilisée pour cette version : interface React, API FastAPI, moteur Rhai/Wasm, schéma PostgreSQL et configuration Docker présents dans le dépôt au 19 juin 2026.